

Fête de La Science

Service web

Rapport de programmation

Audin Lucas Almamy, Kobrossly Rayan, Serigne Mbaye,
Zainab Karim, Kassoum Kone, Adrien Chaudron

4A IA2R





20/01/2026

SOMMAIRE

1. Introduction **3**
2. Architecture Technique **3**
2.1 Les Couches de l'Application 3
3. Conception et Modélisation des Données et DevOps **4**
3.1 Diagramme de classe **4**
3.2 Choix de Conception Importants **6**
3.3 Environnement de Développement et Intégration Continue (CI/CD) 6
4. Logique Métier et Interactions **7**
4.1. Diagramme de Séquence : L'Inscription **7**
4.2 Intégration de Services Tiers : Géolocalisation et Cartographie 8
4.3 Interactivité Côté Client et Architecture Hybride 8
5. Fonctionnalités Clés et Sécurité **9**
5.1 Gestion de la Sécurité **9**
5.2 Gestion des Ateliers (Côté Animateur) **9**
5.3 Expérience Utilisateur (UX) et Gestion des Erreurs 9
5.4 Interface Admin 10
6. Difficultés Rencontrées et Solutions **10**
7. Conclusion et Perspectives **11**

1. Introduction

L'organisation d'un événement d'envergure comme la "Fête de la Science" implique une coordination complexe entre plusieurs entités : les laboratoires de recherche qui ouvrent leurs portes, les établissements scolaires qui souhaitent participer, et l'équipe administrative qui supervise l'ensemble. Jusqu'à présent, la gestion de ces interactions pouvait manquer de fluidité. L'objectif n'est pas seulement de remplacer le papier ou les échanges d'emails, mais d'apporter une réelle valeur ajoutée grâce à des fonctionnalités dynamiques comme la validation des demandes en temps réel et la géolocalisation des activités.

C'est pour répondre à ce besoin de modernisation que nous avons conçu et développé une plateforme web dédiée centralisant la gestion des ateliers et des inscriptions.

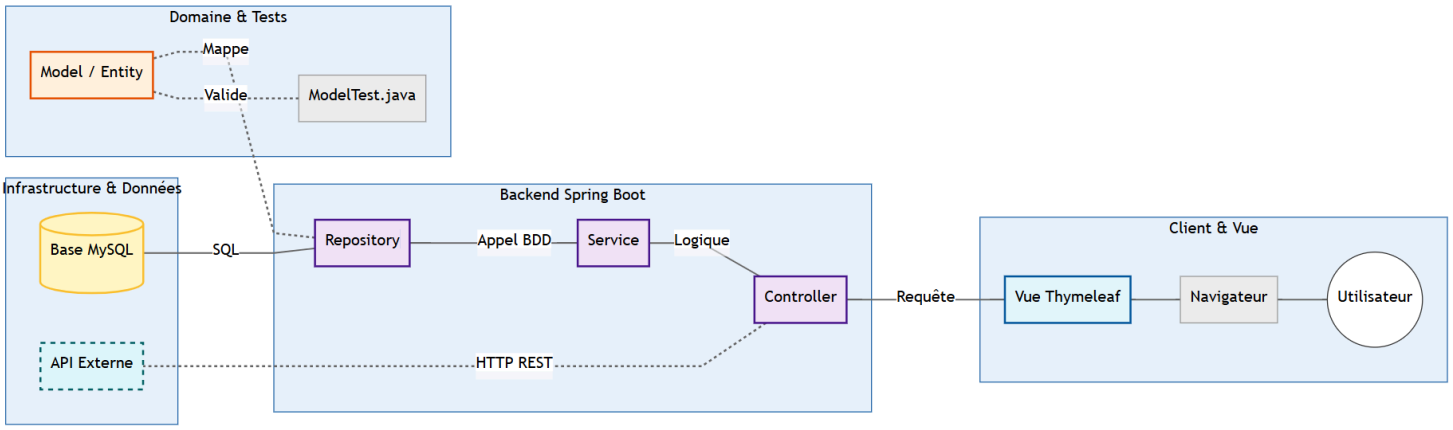
Conçue autour d'une architecture MVC avec Spring Boot et MySQL, l'application structure les interactions entre trois acteurs clés :

- « Les Participants » : Ils consultent le catalogue et s'inscrivent aux créneaux disponibles.
- « Les Animateurs » : Ils créent et gèrent leurs ateliers et les créneaux horaires associés.
- « L'Administrateur » : Il supervise l'ensemble des inscriptions et valide les comptes.

Au-delà de la simple dématérialisation, ce projet intègre des fonctionnalités dynamiques comme la géolocalisation et la validation en temps réel. Ce rapport détaille notre démarche technique, de la modélisation des données à l'industrialisation du code via l'intégration continue.

2. Architecture Technique

Nous avons opté pour une architecture MVC (Modèle-Vue-Contrôleur) stricte, implémentée avec le Framework « Spring Boot ». Cette structure garantit une séparation claire des responsabilités, facilite la maintenance et permet une évolution modulaire du projet.



2.1 Les Couches de l'Application

L'application est divisée en plusieurs couches logiques qui interagissent de manière hiérarchique :

1. Couche Modèle : Elle représente la structure persistante des données. Nous avons utilisé JPA/Hibernate pour le mapping objet-relationnel (ORM).

   Exemple : L'entité Atelier contient les informations descriptives et une liste de créneaux associés.

2. Couche Repository (Data Access) : C'est l'interface directe avec la base de données. Chaque entité possède son repository étendant JpaRepository, ce qui nous donne accès à des méthodes standard (save, findById) tout en permettant des requêtes personnalisées.

   Exemple : InscriptionRepository contient la méthode existsByParticipantAndCreneau pour empêcher les inscriptions en doublon.

3. Couche Service (Business Logic) : C'est le cœur du traitement. Les contrôleurs n'interrogent jamais directement la base de données ; ils sollicitent les services qui appliquent les règles de gestion.

   Exemple : AuthService sécurise l'inscription en hachant les mots de passe via BCrypt avant la sauvegarde.

   Exemple : « ParticipantService » orchestre la récupération des données participants.

4. Objets de Transfert (DTO) : Pour découpler la structure de la base de données (entités) de ce qui est affiché à l'utilisateur, nous utilisons des DTOs (Data Transfer Objects). Cela permet de ne transférer que les données nécessaires.

   Exemple : La classe CreneauDTO est utilisée pour formater les données d'un créneau avant de les envoyer à la vue ou via une API JSON, évitant ainsi d'exposer inutilement toute la complexité de l'entité Creneau

5. Couche Contrôleur : Elle gère les requêtes HTTP entrantes, vérifie les droits d'accès et renvoie les vues correspondantes (pages HTML générées par Thymeleaf).

   Exemple : InscriptionController pilote le flux d'inscription, de l'affichage du formulaire à la redirection finale en cas de succès ou d'erreur.

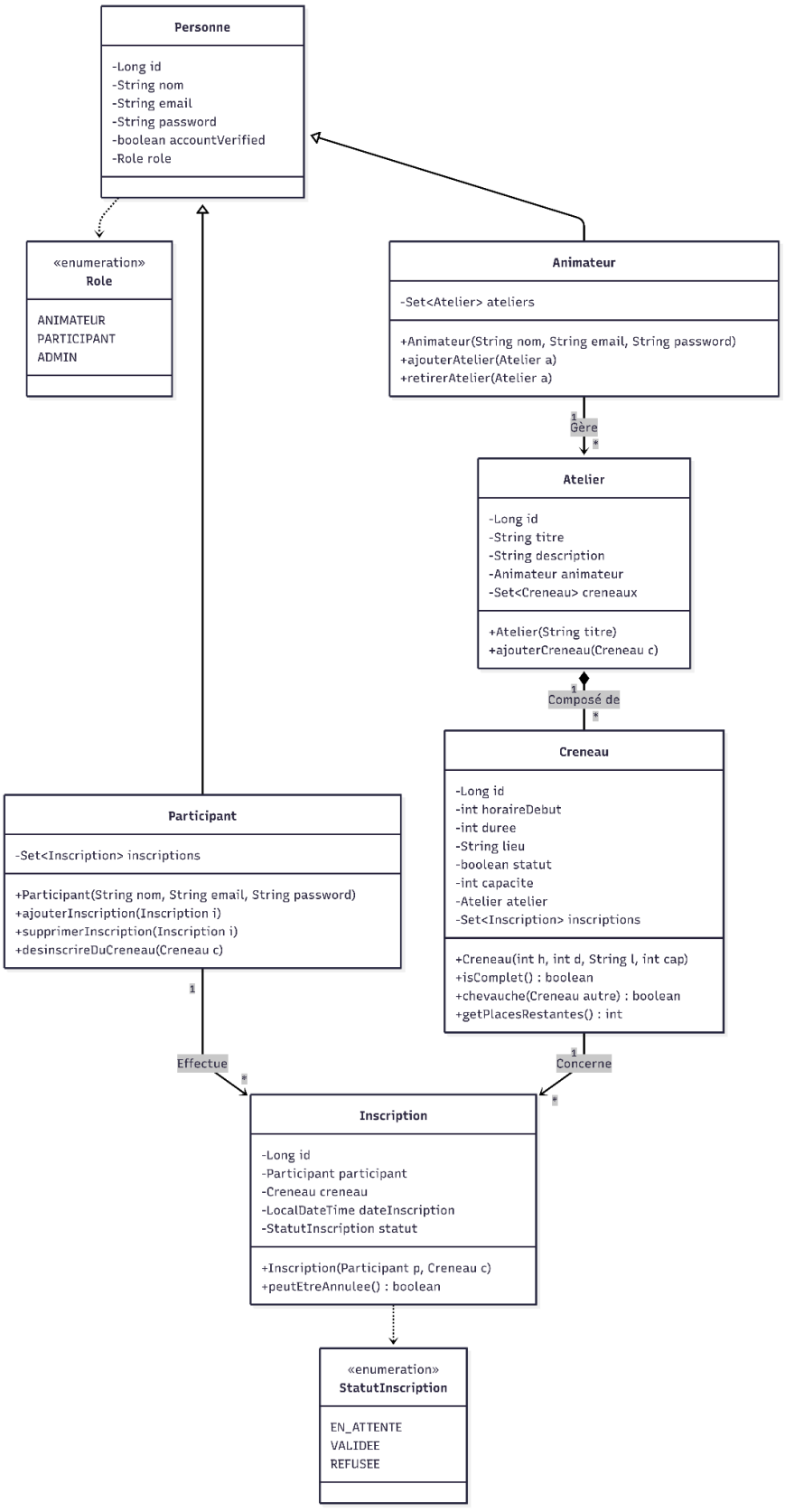
3. Conception et Modélisation des Données et DevOps

3.1 Diagramme de classe

Le diagramme ci-dessus illustre cette séparation structurelle. On y visualise clairement comment les entités comme Animateur ou Participant héritent d'une classe socle Personne, centralisant ainsi les données de connexion et le Rôle. Il met également en évidence la relation de composition forte entre Atelier et Creneau : la suppression d'un atelier entraînant logiquement celle de ses créneaux, une règle métier directement traduite dans le code par la propriété orphanRemoval = true.

Un point clé de cette modélisation réside dans l'entité associative Inscription. Plutôt qu'une simple relation directe entre participants et créneaux, nous avons matérialisé ce lien pour y porter des données contextuelles essentielles : la date de la demande et surtout son état via l'énumération

StatutInscription (En attente, Validée, Refusée). Ce choix structurel est indispensable pour gérer efficacement le workflow de validation sans alourdir les tables principales.



3.2 Choix de Conception Importants

Pour répondre aux besoins du projet, nous avons fait des choix techniques précis au niveau des entités :

1. Héritage (Stratégie JOINED) : Nous avons une classe abstraite Personne qui contient les informations communes (nom, email, mot de passe, rôle). Les classes Animateur et Participant héritent de cette classe. Cela permet d'avoir une table unique pour gérer la connexion (login) tout en ayant des tables séparées pour les données spécifiques.

2. Gestion des Créneaux (Composition) : La relation entre Atelier et Creneau est de type "Composition". Nous avons configuré « orphanRemoval = true », ce qui signifie que si un atelier est supprimé, tous ses créneaux sont automatiquement supprimés de la base de données pour maintenir la cohérence.

3. Table d'Association (Inscription) : Plutôt qu'une simple relation Many-to-Many entre Participant et Créneau, nous avons créé une entité Inscription. Cela nous permet de stocker des informations contextuelles comme la dateInscription et le statut (EN_ATTENTE, VALIDEE, REFUSEE).

4. Utilisation de DTO (Data Transfer Objects) : Pour ne pas exposer directement la structure interne de notre base de données vers l'interface utilisateur, nous avons introduit des DTO (comme CreneauDTO). Cela nous permet de filtrer les données envoyées au client (en masquant certaines relations complexes ou données sensibles) et de découpler la couche de présentation de la couche de persistance.

5. Intégration d'API via un Contrôleur Proxy : Concernant l'affichage des cartes, nous avons choisi de ne pas appeler l'API de cartographie directement depuis le navigateur du client. Nous passons par un contrôleur intermédiaire (MapProxyController) côté serveur. Ce choix architectural permet de contourner les problèmes de CORS (Cross-Origin Resource Sharing) et de centraliser la gestion des appels externes, offrant une meilleure maîtrise des flux de données.

3.3 Environnement de Développement et Intégration Continue (CI/CD)

Pour garantir la qualité du code et faciliter le travail en équipe, nous avons mis en place un environnement de développement structuré reposant sur Git. L'ensemble du code source est versionné, ce qui permet de suivre les modifications, de gérer le travail en parallèle et de conserver un historique clair et cohérent tout au long du projet. Même si nous avions pas d'expérience là-dessus, mais nous avons commencé à comprendre le jeu tout au long du projet.

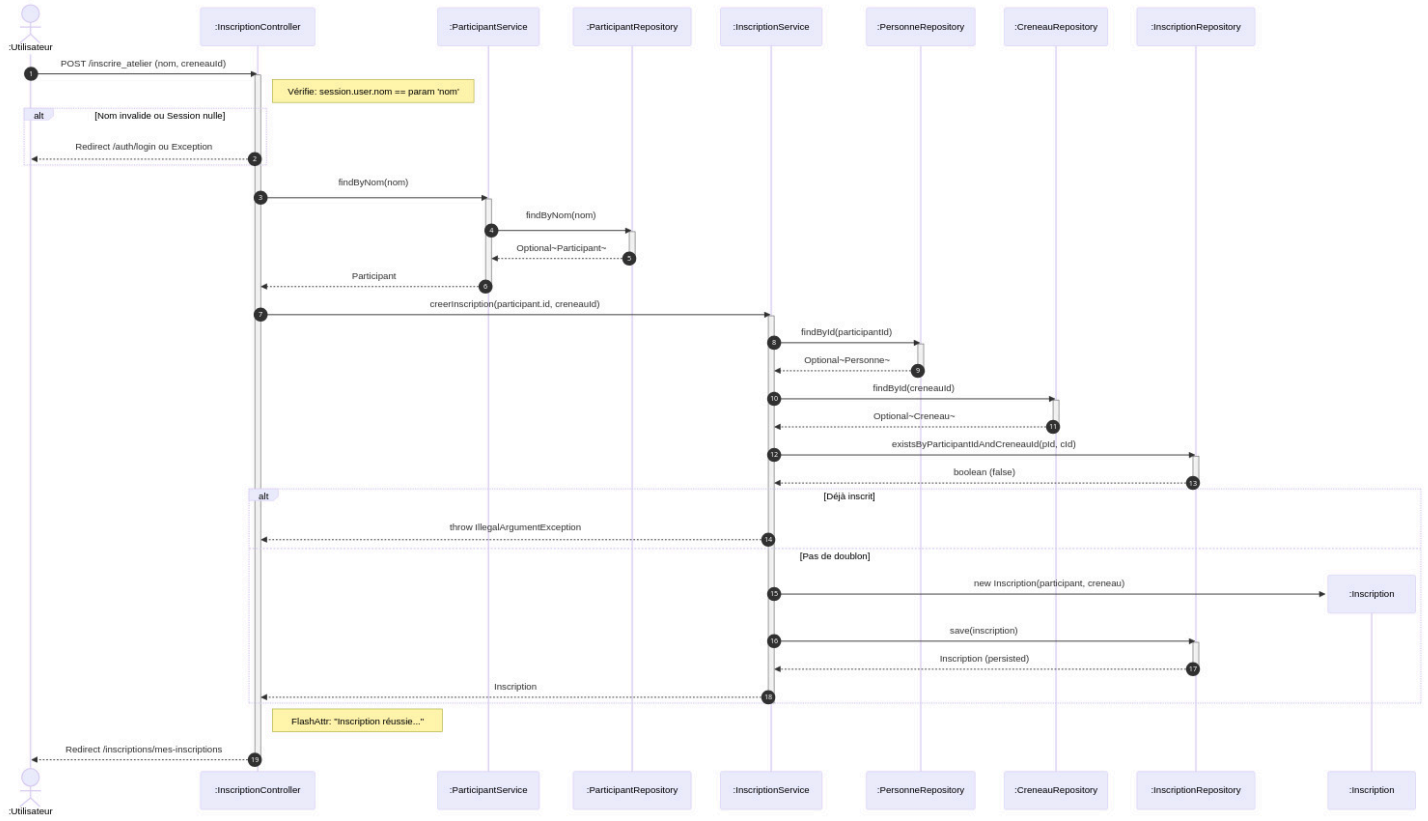
L'automatisation joue également un rôle central grâce à l'utilisation d'un pipeline d'intégration continue avec GitLab CI. Défini dans le fichier .gitlab-ci.yml à la racine du projet, ce pipeline se déclenche automatiquement à chaque mise à jour du dépôt et s'exécute dans un conteneur Docker léger (eclipse-temurin:21-jdk-alpine), garantissant un environnement de build identique pour tous les développeurs.

Le pipeline se compose de deux étapes principales. La phase de build compile le projet et génère l'artefact exécutable via Maven, en s'appuyant sur un système de cache pour accélérer le chargement des dépendances. Une fois cette étape validée, les tests unitaires sont exécutés

automatiquement. En cas d'échec, le pipeline s'arrête immédiatement, évitant ainsi l'intégration de code instable dans la branche principale.

4. Logique Métier et Interactions

4.1. Diagramme de Séquence : L'Inscription



Ce diagramme illustre le parcours d'un utilisateur qui s'inscrit :

1. Le InscriptionController reçoit la requête. Il vérifie d'abord que le nom saisi correspond à l'utilisateur connecté par mesure de sécurité.
2. Le contrôleur appelle ParticipantService pour récupérer l'objet Participant complet.
3. Ensuite, il délègue la création à InscriptionService.
4. Ce service vérifie via le repository si l'utilisateur est déjà inscrit (existsByParticipantIdAndCreneauId) avant de sauvegarder.

4.2 Intégration de Services Tiers : Géolocalisation et Cartographie

Pour garantir la fiabilité des données géographiques et offrir une expérience utilisateur visuelle, nous avons choisi de ne pas gérer la cartographie en interne, mais de nous appuyer sur des services tiers spécialisés. Cette intégration repose sur deux composants majeurs de notre architecture backend.

Validation et Normalisation des Adresses : La qualité des données saisies est critique pour l'organisation logistique de la "Fête de la Science". C'est pourquoi nous avons implémenté le service AddressValidationService. Ce composant agit comme une passerelle technique : lorsqu'un utilisateur saisit une adresse pour un atelier, ce service interroge de manière asynchrone une API de géocodage publique (via RestTemplate ou WebClient). Cette approche nous permet non seulement de vérifier l'existence réelle de l'adresse, mais aussi de récupérer ses coordonnées GPS exactes (latitude et longitude) et de normaliser son format avant l'enregistrement en base de données. Cela évite les erreurs de saisie et garantit une uniformité des données.

Proxy de Cartographie (MapProxyController) : Concernant l'affichage des cartes interactives sur le frontend, nous avons pris le parti technique d'implémenter un contrôleur intermédiaire, le MapProxyController. Plutôt que de laisser le navigateur du client interroger directement le fournisseur de cartes, les requêtes transitent par notre serveur. Ce choix de conception, souvent appelé "Proxy Pattern", répond à deux exigences :

> Sécurité et Encapsulation : Il permet de centraliser les appels vers l'extérieur et, si nécessaire, de masquer les clés d'API ou les tokens d'authentification qui ne doivent pas être exposés dans le code client (JavaScript).

> Gestion du CORS : Il contourne les problèmes fréquents de restrictions d'origine croisée (Cross-Origin Resource Sharing) imposés par les navigateurs, assurant ainsi un chargement fluide des tuiles cartographiques quel que soit l'environnement du client.

4.3 Interactivité Côté Client et Architecture Hybride

L'expérience utilisateur a été fluidifiée grâce à une approche hybride combinant rendu serveur et mises à jour dynamiques côté client.

Rôle de Thymeleaf (Rendu Serveur) : Le moteur de template Thymeleaf assure la structuration initiale des pages. Il génère le code HTML côté serveur en y injectant les données du modèle (attributs de session, informations utilisateur). Cela garantit un affichage rapide du contenu principal et simplifie la gestion de la sécurité (CSRF tokens inclus automatiquement dans les formulaires).

Dynamisme via JavaScript et AJAX: Pour éviter des rechargements de page intempestifs lors d'actions spécifiques, nous avons enrichi l'interface avec du JavaScript. Exemple : Le script charger_creneaux.js interroge le serveur de manière asynchrone (fetch API) pour récupérer et afficher les créneaux disponibles dès qu'un atelier est sélectionné. De même, la suppression d'une inscription se fait sans rafraîchir la page entière.

Une Architecture Backend Bivalente (HTML & JSON) : Cette interactivité repose sur une distinction claire dans nos contrôleurs Spring Boot :

> Les Contrôleurs de Vue (@Controller) : Ils traitent les requêtes de navigation classiques et retournent des chaînes de caractères correspondant aux noms des fichiers HTML (templates Thymeleaf). C'est le cas de InscriptionController qui renvoie la vue du formulaire.

> Les Contrôleurs d'API (@RestController) : Pour les appels AJAX, nous utilisons des endpoints spécifiques qui ne renvoient pas de HTML, mais des données brutes au format JSON. Par exemple, CreneauController expose les disponibilités des ateliers. Cette séparation préfigure notre future architecture : ces endpoints JSON constitueront la base de l'API REST nécessaire à la migration vers un client Angular complet.

5. Fonctionnalités Clés et Sécurité

5.1 Gestion de la Sécurité

La sécurisation de l'application repose intégralement sur le framework Spring Security. L'architecture mise en place s'articule autour de trois axes principaux pour garantir l'intégrité des données et des accès.

Premièrement, la gestion des identités est assurée par le AuthService pour l'inscription et le CustomUserDetailsService pour la connexion. Nous appliquons une politique stricte de confidentialité, aucun mot de passe n'est stocké en clair, ils sont systématiquement hachés via l'algorithme BCrypt avant persistance.

Deuxièmement, nous avons développé une gestion de session personnalisée. Lors d'une connexion réussie, un AuthenticationSuccessHandler injecte l'objet utilisateur en session et orchestre une redirection contextuelle : un administrateur est envoyé vers sa console de gestion tandis qu'un participant rejoint son espace personnel.

Enfin, le cloisonnement des rôles est verrouillé à deux niveaux : globalement via la SecurityFilterChain pour filtrer les requêtes HTTP, et finement au niveau des méthodes des contrôleurs grâce à l'annotation @PreAuthorize("hasRole('ADMIN')"). Cela empêche toute tentative d'accès forcé par manipulation d'URL.

5.2 Gestion des Ateliers (Côté Animateur)

Les animateurs disposent d'un tableau de bord dédié (/animateur_page), sécurisé en amont par Spring Security qui restreint l'accès aux seuls utilisateurs disposant du rôle ROLE_ANIMATEUR.

Cependant, la sécurité ne se limite pas à l'accès aux pages. Nous avons implémenté une vérification de propriété (Owner Check) au cœur de la logique métier. Concrètement, lorsqu'un animateur souhaite modifier ou supprimer un atelier, le système ne se contente pas de vérifier ses droits d'administration globaux. Le contrôleur s'assure systématiquement que l'utilisateur connecté est bien le créateur de l'atelier ciblé.

Cette isolation est gérée dès la récupération des données. Plutôt que de filtrer une liste globale côté client, nous utilisons des méthodes spécifiques du repository (telles que findByAnimateur) pour ne remonter depuis la base de données que les ateliers appartenant strictement à l'utilisateur courant. Cela garantit qu'aucun animateur ne peut, accidentellement ou malicieusement, altérer les événements organisés par un confrère.

5.3 Expérience Utilisateur (UX) et Gestion des Erreurs

L'adoption de la plateforme par les enseignants dépend grandement de la fluidité de leur parcours. Nous avons donc particulièrement soigné les interactions clés du côté participant.

Parcours d'Inscription et de Désinscription : L'interface a été conçue pour offrir une autonomie totale aux participants. Sur le catalogue des ateliers, l'inscription à un créneau se fait en quelques clics, avec une vérification immédiate de la disponibilité. Mais c'est surtout sur la gestion des annulations que l'UX a été optimisée : via l'espace "Mes Inscriptions", un participant peut se désinscrire d'un atelier. Cette action est traitée de manière asynchrone par nos scripts JavaScript (deleteInscription.js), ce qui permet de mettre à jour la liste instantanément sans recharger toute la page, offrant une sensation de réactivité immédiate.

Gestion Personnalisée des Erreurs (403/404) : Pour maintenir une immersion cohérente, nous avons banni les pages d'erreur génériques ("Whitelabel Error Page") de Spring Boot. Lorsqu'un utilisateur tente d'accéder à une ressource inexistante ou interdite, il est redirigé vers des pages spécifiquement designées :

> Erreur 403 (Accès Interdit)

> Erreur 404 (Page Introuvable) : En cas d'URL erronée, une page personnalisée (illustrée graphiquement pour dédramatiser l'erreur) guide l'utilisateur pour qu'il ne se retrouve jamais dans une impasse.

5.4 Interface Admin

L'interface d'administration a été conçue comme une tour de contrôle permettant une vision exhaustive de l'événement. Elle s'articule autour de deux axes majeurs : la gestion structurelle de l'offre (Ateliers/Créneaux) et la régulation de la demande (Inscriptions).

Gestion Centralisée des Ateliers et Créneaux : Via le contrôleur dédié AdminAtelierController, l'administrateur dispose d'un pouvoir de modification total sur le catalogue (CRUD). Il peut créer des ateliers, leur assigner des animateurs spécifiques et définir les créneaux horaires associés. Un point d'honneur a été mis sur l'intégrité des données. Contrairement à une suppression naïve, le système intègre des garde-fous logiques pour éviter les incohérences :

Modification sécurisée : Il est impossible de réduire la capacité d'un créneau en dessous du nombre d'inscrits actuels.

Suppression conditionnelle : Le système bloque la suppression d'un créneau si celui-ci contient des inscriptions actives, forçant l'administrateur à gérer les participants avant de détruire la ressource.

Workflow de Validation des Inscriptions : En parallèle, le tableau de bord des inscriptions permet de fluidifier le traitement des demandes. Plutôt que de naviguer de page en page, l'administrateur visualise les demandes en attente sous forme de liste synthétique. Les actions "Valider" ou "Refuser" déclenchent des mises à jour immédiates en base de données. L'expérience utilisateur est ici renforcée par l'usage des Flash Attributes (RedirectAttributes) : chaque action (ajout de créneau, validation d'inscription) est confirmée par une notification éphémère (succès ou erreur) affichée en haut de page, garantissant à l'administrateur que sa commande a bien été prise en compte par le serveur.

6. Difficultés Rencontrées et Solutions

Ce projet a été pour nous une véritable montée en compétence technique, accompagnée de plusieurs défis structurels que nous avons dû surmonter.

Apprentissage de la Stack Technique La première difficulté majeure a été la prise en main simultanée de trois technologies robustes : Spring Boot, MySQL et Git. Au démarrage, la configuration de l'environnement et la compréhension de l'injection de dépendances de Spring n'étaient pas intuitives. De même, la gestion des conflits de fusion (merge conflicts) sur Git a ralenti nos premiers sprints. Nous avons résolu ces blocages par une communication accrue et la mise en place de règles de commit plus strictes, stabilisant ainsi notre flux de travail.

Rigueur de l'Architecture MVC : Maintenir une séparation stricte entre les couches a demandé une discipline constante. La tentation était parfois grande d'écrire de la logique métier (comme des calculs de dates ou des vérifications de capacité) directement dans les contrôleurs pour gagner du temps. Nous nous sommes forcés de refactoriser systématiquement ces bouts de code vers les classes Service appropriées. Cela a permis de garder des contrôleurs simples, dont le rôle se limite à l'aiguillage des requêtes HTTP.

Gestion de la Densité des Contrôleurs : À mesure que le projet avançait, certains contrôleurs, notamment ceux gérant l'administration, sont devenus massifs avec une multiplication des méthodes @GetMapping et @PostMapping. Pour éviter de créer des fichiers illisibles, nous avons scindé les responsabilités. Par exemple, la gestion des ateliers a été séparée de la gestion globale dans un contrôleur dédié (AdminAtelierController), clarifiant ainsi les routes et la maintenance.

Refactoring et Dette Technique (Sécurité & JS) : Enfin, nous avons dû réécrire (refactorer) des parties entières du code initial.

> Côté Sécurité : La première implémentation de l'authentification était basique. Nous avons dû repenser l'architecture pour intégrer Spring Security proprement, avec un hachage des mots de passe et des gestionnaires de succès personnalisés (SuccessHandler) pour rediriger les utilisateurs selon leur rôle.

> Côté Frontend : Le JavaScript, initialement écrit directement dans les balises <script> des fichiers HTML, devenait difficile à déboguer. Nous avons extrait cette logique dans des fichiers externes (ex: inscriptions.js), rendant le code plus modulaire et réutilisable.

7. Conclusion et Perspectives

La réalisation de la plateforme Fête de la Science nous a permis de confronter la théorie à la réalité du développement d'une application web complexe. Au-delà de la simple réponse au cahier des charges, nous avons eu à cœur de construire une solution robuste et maintenable.

L'architecture mise en place, basée sur le framework Spring Boot et le pattern MVC, a prouvé son efficacité pour structurer le code et sécuriser les échanges de données. Nous avons réussi à dépasser les difficultés initiales liées à la configuration de Spring Security et à la modélisation des données (notamment la gestion fine des créneaux) pour livrer une application fonctionnelle. L'intégration d'outils professionnels comme le pipeline CI/CD sous GitLab et la validation d'adresses via API externe témoigne de notre volonté d'adopter des standards de qualité industrielle dès la phase de développement.

Cependant, nous considérons cette version actuelle comme une étape transitoire. Notre ambition technique vise à faire évoluer ce "monolithe modulaire" vers une architecture distribuée plus moderne:

> Vers une Single Page Application (SPA) : L'objectif prioritaire est de remplacer le moteur de template Thymeleaf par un framework frontend réactif comme Angular. Le code backend a déjà été préparé en ce sens, avec l'exposition progressive d'endpoints JSON via nos @RestController.

> API Restful et Sécurité Stateless : Cette migration impliquera de transformer notre gestion de session actuelle (basée sur les cookies) vers une authentification sans état utilisant des JWT (JSON Web Tokens), rendant l'API totalement indépendante du client.

> Déploiement Conteneurisé : Enfin, pour faciliter l'hébergement et la scalabilité, nous prévoyons d'encapsuler l'application et sa base de données dans des conteneurs Docker, orchestrés via Docker Compose, assurant ainsi un déploiement iso-prod immédiat.